Team Rating Platform — Product & Technical Specification

Document type: Product Requirements + Technical Design **Audience:** Development team, production/DevOps team, QA **Version:** 1.1 (Draft for build) **Owner:** Deekshit Kumar Katikaneni

---

1. Overview

The Team Rating Platform is a lightweight web application for live events (pitch rounds, hackathons, tournaments). Teams register themselves on the site to receive a login, and the number of teams is not fixed. After signing in, a team rates every *other* team on a scale of **1 to 10**. To force careful judgement, each team may award the top score of **10 to at most five teams**. Ratings from all teams are collected, averaged, and shown on a **separate live leaderboard**.

The product has to be simple to use, hard to break, and hard to cheat. It will run during a live event, so reliability matters more than feature count.

1.1 Goals

- Let teams register themselves, then rate every other team quickly (target: under 10 seconds per team rated).
- Enforce the fairness rules on the **server**, never only in the browser.
- Calculate and display a leaderboard automatically as ratings arrive.
- Be production-ready: tested, monitored, deployable through a pipeline, and documented.
- Carry the company branding: embedded logo and a red DyeWhorl visual effect.

1.2 Non-Goals (v1)

- Public voting or audience voting.
- Written feedback or comments.
- Multiple events running at the same time (design allows it later via an `event_id`).
- Payments, emails, or social login.

---

2. Roles

Role	Description	Access
Team	One account per approved team (any number; 30 is a typical event)	Rating page, own submission status, leaderboard (when visible)
Admin	Event organizer	Admin dashboard: manage teams, open/close rating, reset credentials, view audit log, export results
Public viewer (optional)	Projector/screen display	Read-only leaderboard URL

---

3. Business Rules (Source of Truth)

These rules must be enforced in the backend and the database. The UI only mirrors them.

#	Rule
R1	Ratings are **whole numbers from 1 to 10**.
R2	A team **cannot rate itself**.
R3	A team rates each other team **at most once** (one rating per rater–target pair).
R4	A team may give the score **10 to at most 5 teams**.
R5	A team's submission is **final once submitted** (locked). Admin may unlock a team if needed.
R6	Ratings are accepted only while the event status is **OPEN**.
R7	A team's score = **average of all ratings it received** from submitted teams.
R8	The leaderboard updates automatically as submissions come in.

#	Rule
R9	Teams marked **disqualified / withdrawn** are excluded from receiving and giving ratings.
R10	Teams register themselves during the REGISTRATION phase. The admin approves or rejects each registration, and only APPROVED teams can log in, rate, or be rated.
R11	The team list is frozen when rating opens: no team is added and none disappears from the roster. Afterwards the admin can only disqualify (R9).
R12	The number of teams (N) is variable. Each team rates N-1 teams. The cap on 10s is max_tens (default 5), limited to N-1, and rating cannot open until N reaches the minimum (default 3).

> **Why the 10-cap matters:** with no limit, every team gives everyone a 10 and the leaderboard becomes meaningless. The cap forces teams to rank.

---

4. User Flows

4.1 Registration flow

1. Admin sets the event to the **REGISTRATION** phase and shares the registration link or QR code.
2. A team lead opens the link and fills in the form: team name, leader name, email or phone, and a password.
3. The system checks that the team name is unique and creates the team as **PENDING**. It shows the team's **team code** (its login ID) and the message "Awaiting approval".
4. The admin reviews the pending list and approves or rejects each team, one by one or in bulk.
5. When registration closes, the admin **freezes the roster**. The number of approved teams becomes N for this event.
6. Rating opens. Approved teams log in with their team code and password.

4.2 Team flow

1. Team opens the site and sees the branded login screen.
2. Team logs in with its team code and the password it chose at registration.
3. Team lands on the **Rating page**: a list of every other team (self is not shown).
4. Team assigns a rating from 1 to 10 for each team. Progress is autosaved as a draft.
5. A live counter shows **"10s used: X / 5"**. When X reaches 5, the 10 option is disabled on remaining teams, with a clear message.
6. Team clicks **Review & Submit**, sees a summary, confirms.
7. Submission is locked. Team sees a confirmation screen with a link to the leaderboard (if visible).

4.3 Leaderboard flow

1. Anyone with access opens the **Leaderboard page**.
2. Page shows ranked teams with score, rank, and number of ratings received.
3. Page refreshes automatically (realtime or polling) as new submissions arrive.
4. Header shows progress, for example **"20 of N teams have submitted (N = approved teams)"**.

4.4 Admin flow

1. Admin logs in via a separate admin login.
2. Admin creates the event, sets the registration window, approves or rejects registered teams, and freezes the roster before rating opens (manual add and CSV import remain available).
3. Admin sets event status: **DRAFT → REGISTRATION → OPEN → CLOSED**.
4. Admin monitors submission progress, can unlock a team, disqualify a team, toggle leaderboard visibility, and export results.

---

5. Functional Requirements

5.1 Authentication

- FR-A1: Teams self-register (see 5.5) and can log in only after the admin approves them. Login = the team code issued at registration + the password the team chose. Admins can also add teams manually.

- FR-A2: Passwords are stored hashed (bcrypt or argon2). Never stored or logged in plain text.
- FR-A3: Sessions use secure, HTTP-only cookies or short-lived JWTs with expiry.
- FR-A4: Login attempts are rate limited (e.g. 5 failures per 10 minutes per IP and per team code).
- FR-A5: A team can only act as itself. The rater identity is **always taken from the session**, never from the request body.

5.2 Rating page

- FR-R1: Lists all approved teams on the frozen roster except the logged-in team.
- FR-R2: Each team has a 1–10 selector (tap-friendly buttons or a slider; buttons recommended for mobile).
- FR-R3: Live counter of 10s used; the 10 option disables after five.
- FR-R4: Autosave of drafts so a refresh or network drop loses nothing.
- FR-R5: Submit is enabled only when **all** other teams are rated.
- FR-R6: Review screen before final submit; submission is locked afterwards.
- FR-R7: Clear errors for closed event, expired session, and rule violations.

5.3 Leaderboard page

- FR-L1: Ranked list sorted by average score (descending).
- FR-L2: Columns: Rank, Team, Average score (2 decimals), Ratings received.
- FR-L3: Realtime or 5-second polling updates, with a smooth reorder animation.
- FR-L4: Admin can hide or show the leaderboard (useful to reveal results on stage).
- FR-L5: Top 3 are highlighted. Ties share a rank (see 7.2).

5.4 Admin dashboard

- FR-D1: Team management: add, edit, import CSV, regenerate password, disqualify.
- FR-D2: Event control: DRAFT / REGISTRATION / OPEN / CLOSED, roster freeze, leaderboard visibility toggle.
- FR-D3: Live submission tracker (who has submitted, who has not).
- FR-D4: Unlock a team's submission (logged in the audit trail).
- FR-D5: Export results as CSV: leaderboard plus the full rating matrix.
- FR-D6: Audit log viewer.

5.5 Team registration

- FR-G1: The public registration page is available only in the REGISTRATION phase; otherwise it shows "Registration closed".
- FR-G2: Form fields: team name (unique per event, 2 to 40 characters), leader name, email or phone, and a password of at least 8 characters.
- FR-G3: On success the team receives a short **team code** (its login ID) and the status **PENDING**. The code is shown on screen once.
- FR-G4: The form is protected against abuse: rate limiting per IP, a bot check (CAPTCHA or similar), and an optional event invite code.
- FR-G5: The admin sees a pending queue with approve, reject (with a reason) and bulk approve. Rejected teams cannot log in.
- FR-G6: Freezing the roster fixes N as the count of APPROVED teams. Opening rating is blocked while N is below the minimum (default 3).
- FR-G7: Contact details are visible to admins only, never to other teams.

---

6. Architecture

6.1 Recommended stack

The stack below matches common production practice and is simple to operate. The team may substitute equivalents, but the **rules in Section 3 must still be enforced server-side**.

Layer	Recommendation
Frontend	Next.js (React) + TypeScript + Tailwind CSS
Backend	Next.js API routes / server actions (or a small Node service)
Database	PostgreSQL (e.g. Supabase)
Realtime	Supabase Realtime, or polling fallback every 5 seconds
Hosting	Vercel (app) + managed Postgres
Monitoring	Sentry (errors) + uptime check + platform logs
CI/CD	GitHub Actions

6.2 High-level flow

Browser (Team) → **API layer** (auth, validation) → **Database** (constraints, aggregate view) → **Leaderboard view** → **Browser (Leaderboard)**

6.3 Pipeline principle

Every rating travels through one fixed, testable pipeline:

1. **Authenticate** — identify the team from the session.
2. **Authorize** — event is OPEN, team is active and not already locked.
3. **Validate** — payload shape, values 1–10, no self-rating, no duplicates, all targets exist.
4. **Enforce** — 10-cap check (R4).
5. **Persist** — write all ratings and lock the submission in **one database transaction**.
6. **Aggregate** — leaderboard recalculates from stored ratings.
7. **Broadcast** — leaderboard clients receive the update.
8. **Audit** — action recorded.

If any step fails, nothing is saved and the team gets a clear error. Partial writes must be impossible.

---

7. Data Design

7.1 Tables

events

Column	Type	Notes
id	uuid (PK)	
name	text	
status	enum	`DRAFT`, REGISTRATION, `OPEN`, `CLOSED`
leaderboard_visible	boolean	default false
max_tens	int	default 5 (configurable)
created_at	timestamptz	
registration_closes_at	timestamptz, nullable	Registration ends automatically at this time
min_teams	int	Default 3. Rating cannot open below this
roster_frozen_at	timestamptz, nullable	Set when the admin freezes the roster; N = approved teams at that moment

teams

Column	Type	Notes
id	uuid (PK)	
event_id	uuid (FK)	
code	text, unique per event	login identifier
name	text	display name
password_hash	text	
status	enum	`PENDING, REJECTED, APPROVED`, `DISQUALIFIED`, `WITHDRAWN`

Column	Type	Notes
submitted_at	timestamptz, nullable	set on final submit (lock)
created_at	timestamptz	
leader_name	text	Team lead's name, from registration
contact	text	Email or phone; visible to admins only
registered_at	timestamptz	When the team registered
approved_at	timestamptz, nullable	Set when the admin approves the team

ratings

Column	Type	Notes
id	uuid (PK)	
event_id	uuid (FK)	
rater_team_id	uuid (FK)	
target_team_id	uuid (FK)	
score	smallint	
status	enum	`DRAFT`, `FINAL`
updated_at	timestamptz	

audit_log

Column	Type	Notes
id	bigserial (PK)	
actor	text	team code or admin id
action	text	e.g. `LOGIN`, `SUBMIT`, `UNLOCK`, `STATUS_CHANGE`
detail	jsonb	

Column	Type	Notes
created_at	timestamptz	

7.2 Database constraints (safety net)

- `CHECK (score BETWEEN 1 AND 10)`
- `CHECK (rater_team_id <> target_team_id)` — enforces R2 at the lowest level.
- `UNIQUE (rater_team_id, target_team_id)` — enforces R3.
- A trigger or the submit function enforces R4: count of `score = 10` per rater must be ⩽ `max_tens`.
- Foreign keys on all references.

7.3 Leaderboard calculation

A database **view** (or materialized view refreshed on submit) computes:

- `avg_score` = average of `FINAL` ratings received by the team
- `ratings_received` = count of those ratings
- `rank` using dense ranking by `avg_score` descending

Tie-breakers (in order):

1. Higher count of 10s received
2. Higher count of 9s received
3. Otherwise share the same rank

Only `FINAL` ratings count. Drafts never affect the leaderboard. Ratings given **by** or **to** disqualified teams are excluded (R9).

> **Fairness note:** because each team cannot rate itself, every team is rated by at most N-1 others. Average is used (not sum) so that teams are not penalised while others have not yet submitted. The "ratings received" column makes partial data visible.

---

8. API Specification

All endpoints return JSON. All write endpoints require an authenticated session and are validated server-side.

Method	Endpoint	Purpose	Auth
POST	`/api/auth/login`	Team or admin login	Public
POST	`/api/auth/logout`	End session	Team/Admin
GET	`/api/teams/targets`	List teams the current team may rate (excludes self)	Team
GET	`/api/ratings/me`	Current draft or final ratings and 10s used	Team
PUT	`/api/ratings/draft`	Autosave one or more draft ratings	Team
POST	`/api/ratings/submit`	Final submit (atomic, locks submission)	Team
GET	`/api/leaderboard`	Ranked leaderboard and submission progress	Team/Admin/Public if enabled
GET	`/api/admin/teams`	List teams and submission status	Admin
POST	`/api/admin/teams/import`	CSV import	Admin
POST	`/api/admin/teams/:id/unlock`	Unlock submission	Admin
POST	`/api/admin/teams/:id/disqualify`	Disqualify team	Admin
POST	`/api/admin/event/status`	Set DRAFT / REGISTRATION / OPEN / CLOSED	Admin

Method	Endpoint	Purpose	Auth
POST	`/api/admin/leaderboard/visibility`	Show/hide leaderboard	Admin
GET	`/api/admin/export`	CSV export of results	Admin
POST	/api/teams/register	Register a team (status PENDING)	Public, REGISTRATION phase only
POST	/api/admin/teams/:id/approve	Approve a registration	Admin
POST	/api/admin/teams/:id/reject	Reject a registration (with a reason)	Admin
POST	/api/admin/event/freeze-roster	Freeze the roster and fix N	Admin

8.1 Submit request example

```
POST /api/ratings/submit
{
  "ratings": [
    { "targetTeamId": "uuid-1", "score": 8 },
    { "targetTeamId": "uuid-2", "score": 10 }
  ]
}
```

The rater is **never** sent by the client. It is read from the session.

8.2 Standard error codes

Code	Meaning
`EVENT_NOT_OPEN`	Rating is closed or not yet started
`ALREADY_SUBMITTED`	Submission is locked
`SELF_RATING`	Attempt to rate own team

Code	Meaning
`INVALID_SCORE`	Score outside 1–10
`TOO_MANY_TENS`	More than the allowed number of 10s
`INCOMPLETE`	Not all target teams were rated
`UNAUTHORIZED`	Not logged in or session expired
`RATE_LIMITED`	Too many requests
REGISTRATION_CLOSED	Registration is not open
TEAM_NAME_TAKEN	A team with this name is already registered for the event
NOT_APPROVED	The team is pending, rejected, or disqualified
ROSTER_NOT_FROZEN	Rating cannot open until the roster is frozen and N meets the minimum

---

9. UI & Branding

9.1 Logo

- The company logo (supplied by the owner as SVG/PNG, transparent background) appears on the login screen, the page header, and the leaderboard header.
- Provide an SVG where possible for sharp rendering. Minimum PNG size: 512 px wide.

9.2 DyeWhorl effect (red)

The brand background is **DyeWhorl**, a real-time fluid simulation that draws red ink drifting and curling through dark water. It moves on its own with no input, and it reacts when a visitor drags a mouse or finger across it. The component is supplied as a single file, `dye-whorl.tsx`, and is used as delivered except for the one-line change under "Red theme" below.

Where it appears

Page	Treatment	Reason
Login	Full screen, interactive, wraps the login card (logo and form)	First impression; the page does not scroll
Leaderboard	Fixed background layer, running, not interactive	Readable from a distance on a projector; the list must scroll
Rating page	Fixed background layer, `paused` (still frame)	Long list on phones; saves battery and keeps focus on scoring
Admin	None	Utility pages

Project prerequisites

The component expects a shadcn-style project with Tailwind CSS and TypeScript. The shadcn default component folder is `/components/ui`. Keep it: the import `@/components/ui/dye-whorl` depends on that folder and the `@/` alias, and components added later with the shadcn CLI land there too. If the project has none of this yet, create the app with the TypeScript option (for Next.js, `create-next-app --ts`) and run:

```
npx shadcn@latest init
npm install tailwindcss @tailwindcss/postcss postcss tw-animate-css   # only if Tailwind 4 is missing
```

Files to add

File	Location	Notes
`dye-whorl.tsx`	`components/ui/dye-whorl.tsx`	Supplied by the owner. Starts with `"use client"`. No extra npm packages
Theme variables	`app/globals.css` (Tailwind 4: the main CSS file; Tailwind 3: `globals.css` or `tailwind.config.js`)	Add `--ns-muted` and `--ns-accent`, plus the red override below
`demo.tsx`	Not shipped	Usage reference only

Base variables the component needs:

```
@import "tailwindcss";
@import "tw-animate-css";

:root {
  --ns-muted: #4d4d4d;
  --ns-accent: #006bff;
}

.dark {
  --ns-muted: #8f8f8f;
}
```

Red theme

The component paints its ink from five CSS variables: `--background` (empty water), `--border`, `--ns-muted`, `--foreground` (dense ink) and `--ns-accent` (a faint tint where a visitor touches the ink). It reads them with `getComputedStyle` and accepts **only 3- or 6-digit hex values**. Current shadcn and Tailwind 4 templates often define colours as `oklch(...)`; those fail to parse and the effect silently falls back to grey and blue. Define the red palette in hex:

```
.dye-red {
  --background: #0b0507;  /* near-black water */
  --border:     #4a0a14;  /* faint crimson haze */
  --ns-muted:   #a3121f;  /* mid ink */
  --foreground: #ff2e3f;  /* dense ink */
  --ns-accent:  #ff6b5e;  /* touch tint */
}
```

These values are a starting point for the owner to approve. The densest cores lighten toward pink by design, because the component blends its top stop toward white.

By default the component reads these variables from the page root, which would turn all site text red. Avoid that with one change inside `dye-whorl.tsx`, in `readColors`: replace `getComputedStyle(document.documentElement)` with `getComputedStyle(wrap)`. The variables are then read from the component's own wrapper, so the `dye-red` class on the component sets the colours for the effect only. Leave a code comment marking this change so it survives future updates of the component.

Usage

```
import DyeWhorl from "@/components/ui/dye-whorl";

// Login: wraps the card
<DyeWhorl className="dye-red h-screen w-full">
  <main className="flex h-full items-center justify-center p-6">
    <LoginCard />
  </main>
</DyeWhorl>

// Leaderboard (running) and rating page (paused): background layer behind
scrolling content
const layer = { position: "fixed", inset: 0, zIndex: -1, pointerEvents:
"none" } as const;
<DyeWhorl className="dye-red" style={layer} />            // leaderboard
<DyeWhorl className="dye-red" style={layer} paused />      // rating page
```

The layer is positioned through `style` because the component's own `relative` class would override a `fixed` utility class. No ancestor may paint an opaque background over it.

Prop	Default	Guidance
`speed`	1	Keep constant. Changing it at runtime rebuilds the WebGL context
`density`	1	Ink added per second, 0 to 2. Use 0.6 to 0.8 on pages with a lot of text
`stir`	1	Pointer strength, 0 to 2. Only matters on the interactive login page
`paused`	false	Shows a fully developed still frame. Can be switched while the page is open
`className`, `style`	none	Size and position; add `dye-red` here
`children`	none	Content drawn above the effect

Rules for developers

- **One instance per page.** Each instance owns a WebGL context.

- **Never wrap scrolling content.** The wrapper sets `touch-none` and listens for pointer events, so scrolling inside it breaks on phones. Use the fixed-layer pattern for any page that scrolls.
- **Readable text.** Place text on a panel such as `bg-black/55 backdrop-blur`, and check WCAG AA contrast on the panel, not on the animation.
- **WebGL2 required.** The effect needs WebGL2 with float render targets. Where unavailable it draws nothing and the wrapper's `--background` shows, which under `dye-red` is the near-black base. Add a crimson gradient class as a visible fallback.
- **Performance manages itself.** The component lowers its own resolution in four steps if frames run slow, pauses when off screen or in a hidden tab, and shows a still frame when the visitor prefers reduced motion. Do not add a second throttle.
- **Logo.** Place the supplied SVG logo inside the card on the login page and in the header of the leaderboard, about 56 px high at most, in white or light red for contrast.
- **Cleanup.** The component releases its GPU resources on unmount. Verify across route changes that the number of WebGL contexts does not grow.

9.3 Layout guidelines

- Mobile-first. Teams will most likely rate from phones.
- Rating rows: team name on the left, 1–10 buttons on the right (wrapping on small screens).
- A sticky footer bar shows **progress (e.g. 12/29 rated)**, **10s used (X/5)**, and the Submit button.
- Leaderboard is projector-friendly: large type, high contrast, auto-updating.

---

10. Non-Functional Requirements

Area	Requirement
Availability	99.9% during the event window
Performance	Page loads under 2 s on 4G; API responses under 300 ms (p95)
Concurrency	At least 300 simultaneous users; all registered teams (design target: up to 100) submitting within the same minute without errors

Area	Requirement
Data integrity	Submit is atomic; no partial or duplicate ratings
Security	HTTPS only, hashed passwords, CSRF protection, input validation, rate limiting, least-privilege DB access
Privacy	Individual ratings are visible **only to admins**. Teams never see who rated them what.
Compatibility	Latest Chrome, Safari, Edge, Firefox; iOS and Android browsers
Accessibility	Keyboard navigable, WCAG AA contrast, reduced-motion support

---

11. Reliability & Edge Cases

Scenario	Expected behaviour
Team refreshes or loses connection mid-rating	Draft is restored from the server
Team double-clicks Submit	Second request rejected as `ALREADY_SUBMITTED`; no duplicates (idempotent)
Team tries to rate itself via API tampering	Rejected by API validation and the DB constraint
Team tries to give six 10s via API tampering	Rejected with `TOO_MANY_TENS`; nothing saved
Event closed while a team is submitting	Rejected with `EVENT_NOT_OPEN`
A team is disqualified after others rated it	Its ratings given and received are excluded; leaderboard recalculates
Team not rated by everyone yet	Average is calculated from available ratings; count shown

Scenario	Expected behaviour
Two teams tie	Tie-breakers applied (7.3), otherwise shared rank
Realtime connection drops	Client falls back to polling automatically
Admin unlocks a team	Team can edit and resubmit; action logged
Team registers after registration closes	Rejected with REGISTRATION_CLOSED; before the freeze the admin may add the team manually
Team is still PENDING when rating opens	Cannot log in, is not on the roster, and is not rated
Admin opens rating without freezing the roster	Blocked with ROSTER_NOT_FROZEN
Very few teams (N-1 is not above the 10 cap)	Cap is limited to N-1; the owner decides whether to scale it (open decision 4)

---

12. Security Checklist

- [ ] All traffic over HTTPS with HSTS
- [ ] Passwords hashed; credentials distributed through a secure channel
- [ ] Server-side validation of every rule (never trust the client)
- [ ] Rater identity derived only from the session
- [ ] Rate limiting on login and write endpoints
- [ ] Admin routes protected by a separate role check
- [ ] Database access via least-privilege roles; secrets in environment variables only
- [ ] Row-level security or equivalent so teams cannot read others' raw ratings
- [ ] Dependency scanning enabled in CI
- [ ] Audit log for logins, submissions, unlocks, and status changes
- [ ] Registration form: rate limiting, bot protection, unique team names, minimum password length

---

13. Environments & Delivery Pipeline

13.1 Environments

Environment	Purpose
Local	Developer machines with a local or dev database
Staging	Mirror of production, used for QA and a full dry run
Production	Live event

Each environment has its own database and secrets. Never share credentials between them.

13.2 CI/CD pipeline

1. **Push / Pull Request** → triggers the pipeline.
2. **Lint & type-check**.
3. **Unit tests** (rules, validation, ranking logic).
4. **Integration tests** (API plus database, including constraint violations).
5. **Build**.
6. **Deploy to Staging** automatically on merge to `main`.
7. **Smoke tests** on Staging.
8. **Manual approval** → **Deploy to Production**.
9. **Post-deploy health check**, with one-click rollback if it fails.

13.3 Database migrations

- All schema changes are versioned migration files, run automatically in the pipeline.
- Migrations must be backward compatible with the previous app version.
- Take a backup before every production migration.

13.4 Configuration (environment variables)

Variable	Purpose
`DATABASE_URL`	Database connection

Variable	Purpose
`SESSION_SECRET`	Session/JWT signing
`ADMIN_BOOTSTRAP_EMAIL` / `ADMIN_BOOTSTRAP_PASSWORD`	First admin creation (rotate after first login)
`MAX_TENS`	Default 5
`RATE_LIMIT_*`	Login and write rate limits
`SENTRY_DSN`	Error reporting
`NEXT_PUBLIC_APP_URL`	Public base URL

---

14. Testing Plan

Level	What to cover
Unit	Score validation, no-self-rating, 10-cap logic, average and rank calculation, tie-breakers
Integration	Each API endpoint, DB constraints rejecting invalid data, atomic submit rollback, session handling
End-to-end	Full team journey: login → rate → submit → leaderboard. Admin journey: import → open → monitor → close → export
Load	100 concurrent users; 30 teams submitting simultaneously
Security	Tamper requests (self-rating, extra 10s, other team's ID), unauthenticated access, brute-force login
UAT / Dry run	Full rehearsal on Staging with test teams before the event

14.1 Acceptance criteria (sign-off)

1. A team can never see itself in its rating list and the API rejects self-rating.

2. A team cannot submit more than the allowed number of 10s (default five), even by bypassing the UI.
3. Submission is atomic, final, and idempotent.
4. The leaderboard ranks correctly and updates within 5 seconds of a submission.
5. If the 19th team has the highest average among the teams that have submitted, it is ranked first.
6. Logo is shown on all main screens and the red DyeWhorl effect renders without hurting readability or performance.
7. All tests pass in CI and a full staging dry run succeeds.
8. Teams can register while registration is open, are blocked after it closes, and cannot log in until approved.
9. Rating opens only after the roster is frozen, and the rating list and leaderboard use exactly the approved teams, for any team count from 3 to 100.

---

15. Monitoring & Operations

- **Error tracking:** all frontend and backend errors to Sentry with alerts to the on-call developer.
- **Uptime:** external health check on `/api/health` every minute.
- **Metrics to watch:** login failures, submit success rate, API latency, DB connections.
- **Event-day runbook:**
  1. T-24h: freeze code, run the staging dry run, take a database backup.
  2. T-1h: verify health checks, confirm admin access, confirm the roster is frozen and every team has its team code.
  3. Event start: set status to **OPEN**, watch the dashboards.
  4. Event end: set status to **CLOSED**, export results, reveal the leaderboard.
  5. Post-event: archive data and rotate or disable credentials.
- **Rollback:** previous deployment can be restored in one click; database restore from the latest backup if required.

---

16. Suggested Folder Structure

```
/app                  # pages and routes (login, rate, leaderboard, admin)
/components           # UI components (RatingRow, TensCounter,
LeaderboardTable, DyeWhorlBackground)
/lib
  /rules              # pure functions: validateRatings, countTens,
computeRanking
  /auth               # session, password hashing
  /db                 # queries, transactions
/api                  # route handlers
/migrations           # versioned SQL
/tests                # unit, integration, e2e
/docs                 # this document, runbook, API reference
/.github/workflows    # CI/CD pipeline
```

Design principle: keep business rules in small, pure, well-tested functions inside `/lib/rules`. The API and UI call these functions, so the logic exists in one place.

---

17. Delivery Plan (Indicative)

Phase	Scope	Estimate
1	Project setup, CI/CD, database schema and migrations	1–2 days
2	Auth, team registration and approval, admin basics	2 days
3	Rating page, draft autosave, submit pipeline, rule enforcement	3 days
4	Leaderboard, ranking logic, realtime updates	2 days
5	Branding: logo and DyeWhorl effect, responsive polish	1–2 days
6	Testing, load test, security checks, staging dry run	2–3 days
7	Production deploy, runbook handover	1 day

---

18. Open Decisions (Owner to Confirm)

1. **Registration control:** open self-registration with admin approval (proposed), or an event invite code that approves teams automatically?
2. **Registration fields:** confirm team name, leader name, email or phone, and password (proposed). Are member names needed?
3. **Late teams:** confirm the roster freezes when rating opens (proposed), or may the admin add a team during the event?
4. **The cap on 10s with different team counts:** keep a fixed 5, or scale it with the number of teams?
5. **Leaderboard visibility:** live for everyone during the event, or hidden until the admin reveals it?
6. **Draft vs final:** is the proposed autosave plus final lock acceptable, or should teams be able to edit after submitting?
7. **Tie-breaker order:** confirm the proposed rule (more 10s, then more 9s).
8. **Exact brand colours** (including the red values in 9.2) and the final logo file.
9. **Hosting and domain** preferences for production.

---

19. Glossary

Term	Meaning
Rater	The team currently giving ratings
Target	A team being rated
Draft	Saved but not final ratings
Final / Locked	Submitted ratings that count toward the leaderboard
DyeWhorl	The animated swirling-dye visual effect used for branding, in red
Team code	Short login ID issued to a team at registration
Roster freeze	Admin action that locks the list of approved teams (N) before rating opens

Term	Meaning
PENDING / APPROVED	Registration states: awaiting an admin decision / allowed to take part